

Assignment 1: Camera Calibration

Haeun Jeon (2023100086)

Dept. Computer Science and Engineering, Korea University

Introduction to Computer Vision (DATA302)

Prof. Gyeongsik Moon

Oct. 3, 2025

1. Description of implementation steps

I implemented this by referencing the lecture slides and the paper(Z. Zhang, "A flexible new technique for camera calibration," IEEE TPAMI). First, the classes I implemented and their roles are as follows.

- `class BoardSpec` : Defines the checkboard specifications (number of rows/columns, size of each square).
- `class CornerDetector` : Corner detection and preprocessing
- `class ZhangCalibration` : Implementation of Zhang's method
- `class ZhangRefinement` : Non-linear refinement for Zhang's method
- `class OpenCVCalibration` : Runs camera calibration using OpenCV's `cv2.calibrateCamera()`
- `class Visualizer` : Visualization of results (plots observed vs. reprojected points, undistorted image)

1.1 Load images and detect corners

1.1.1 Load images and define the physical checkerboard specs

- **Implementation** : `class BoardSpec`
- **Role** : Defines the physical specifications of the checkerboard and generates 3D coordinates
- **Features**
 - Uses the same coordinate system as OpenCV (X: horizontal, Y: vertical, Z = 0).
 - Converts to homogeneous coordinates to support various transformations.
 - 3D homogeneous : $M = [X, Y, Z=0, 1]$
 - 2D homogeneous : $p = [u, v, 1]$
 - Defines the size of each square (as in the instructions, 20mm)
 - Defined as rows=13, cols=9, square_size=20(mm)

1.1.2 Corner detection

- **Implementation** : `class CornerDetector`
- **Role** : Detects and refines corners from checkerboard images
- **Features**
 - Uses OpenCV `cv2.findChessboardCorners()`
 - OpenCV `cv.cornerSubPix()` for subpixel refinement to improve accuracy.
 - Converts all detected coordinates to homogeneous form $[u, v, 1]$

1.2. Zhangs Method

- Implementation and Code structure

`class ZhangCalibration` → create object : `ZhangCalibrator.calibrate(p_plane, m_list)` → call : `calib.calibrate()`

- **Overall pipeline** : homography → intrinsics → extrinsics in `calibrate()`

1.2.1 Homography Estimation

- **Implementation** : `def estimate_homographies` , `def _dlt_homography`
- **Method** : Estimate the homography using the DLT algorithm

- Key Features

- **Normalized DLT**: Coordinate normalization for numerical stability
- **SVD-based solution**: Solve the linear system of the form $Ah = 0$
- **Denormalization**: Map results from the normalized coordinate system back to the original coordinates

- Detailed Procedure

1) perspective projection equation : $s\mathbf{m} = \mathbf{A}[\mathbf{R} \ t]\mathbf{M}$. (1)

- m : the detected corner coordinates (homogeneous) contained in `m_list`
- M : 2D image coordinates (homogeneous), i.e., `p_plane`
- A : intrinsic parameter
- $[R \ t]$: extrinsic parameter

2) By rearranging this Eq. (1), if we take x to be the row vectorized form of H , we obtain the expression shown in the figure(Appendix A of the paper). In other words, a linear system of the form $Lx = 0$ is constructed, where L is a known matrix and x is unknown, so it can be solved.

$$\underbrace{\begin{bmatrix} \mathbf{M}^T & \mathbf{0}^T & -u\mathbf{M}^T \\ \mathbf{0}^T & \mathbf{M}^T & -v\mathbf{M}^T \end{bmatrix}}_{L(\mathbf{M}, u, v)} \underbrace{\begin{bmatrix} \mathbf{h}_1 \\ \mathbf{h}_2 \\ \mathbf{h}_3 \end{bmatrix}}_x = \mathbf{0}. \quad (2)$$

3) Substituting $M = [X, Y, 1]$ into the Eq. (2) yields the following Eq. (3)

$$\underbrace{\begin{bmatrix} X & Y & 1 & 0 & 0 & 0 & -uX & -uY & -u \\ 0 & 0 & 0 & X & Y & 1 & -vX & -vY & -v \end{bmatrix}}_{\text{from } (X, Y, 1) \text{ and } (u, v)} \begin{bmatrix} \mathbf{h}_1 \\ \mathbf{h}_2 \\ \mathbf{h}_3 \end{bmatrix} = \mathbf{0}, \quad \text{with } \mathbf{H} = [\mathbf{h}_1 \ \mathbf{h}_2 \ \mathbf{h}_3]. \quad (3)$$

4) If the number of points is N , then using all points turns the matrix in Eq. (3) into a $(2N \times 9)$ matrix. Solving it with SVD yields the previously unknown homography H .

⇒ Return value of `def _dlt_homography()` : homography H

1.2.2. Intrinsic parameter

- **Implementation** : `def _v_ij_from_H()` , `def _stack_V()` , `def _solbe_b()`, `def _A_from_b()`, `def _fix_skew_in_A()`

- **Method** : Build $Vb = 0$ and obtain a closed-form solution. (Although the lecture note mentions using Cholesky, using Cholesky didn't improve performance; I instead follow the closed-form solution introduced in Section 3.1 of the paper.)

- Key Features

- **Zhang's constraints** : Extract intrinsic constraints from homography
- $Vb = 0$ **linear system** : Stack constraints from multiple views to form a linear system.
- **SVD solution** : Solve $Vb = 0$ via SVD
- **Skew fixed** : Set the skew to 0 (same as OpenCV).

(When not fixed to 0, the performance differed significantly from OpenCV.)

- Detailed procedure

1) We have obtained the Homography H , and from Eq. (1) we know $H = A[r_1 \ r_2 \ t]$. Writing this column-wise gives Eq. (4).

$$\mathbf{h}_1 = \mathbf{A} \mathbf{r}_1, \quad \mathbf{h}_2 = \mathbf{A} \mathbf{r}_2, \quad \mathbf{h}_3 = \mathbf{A} \mathbf{t}. \quad (4)$$

2) The two columns r_1, r_2 of the rotation matrix are orthonormal ($\|r_1\| = \|r_2\| = 1$, $r_1^T r_2 = 0$). Expressing these constraints and substituting Eq. (4) yields Eq. (5)

$$\begin{aligned} \mathbf{r}_1^T \mathbf{r}_2 &= 0, \quad \mathbf{r}_1^T \mathbf{r}_1 = \mathbf{r}_2^T \mathbf{r}_2 \quad \text{and let } \mathbf{r}_i = \mathbf{A}^{-1} \mathbf{h}_i. \\ \mathbf{h}_1^T \mathbf{A}^{-T} \mathbf{A}^{-1} \mathbf{h}_2 &= 0, \quad \mathbf{h}_1^T \mathbf{A}^{-T} \mathbf{A}^{-1} \mathbf{h}_1 = \mathbf{h}_2^T \mathbf{A}^{-T} \mathbf{A}^{-1} \mathbf{h}_2. \end{aligned} \quad (5)$$

3) Since $B = A^{-T} A^{-1}$, B is symmetric. According to the paper, if we parameterize A with the intrinsic parameters as in Eq. (6) and substitute, we obtain the relation in Eq. (7). The paper then introduces a vector b and constructs a linear system $Vb = 0$, where each row of V is defined as in Eq. (8).

$$\mathbf{A} = \begin{bmatrix} \alpha & \gamma & u_0 \\ 0 & \beta & v_0 \\ 0 & 0 & 1 \end{bmatrix}. \quad (6)$$

$$\mathbf{B} = \mathbf{A}^{-T} \mathbf{A}^{-1} = \begin{bmatrix} B_{11} & B_{12} & B_{13} \\ B_{12} & B_{22} & B_{23} \\ B_{13} & B_{23} & B_{33} \end{bmatrix} = \begin{bmatrix} \frac{1}{\alpha^2} & -\frac{\gamma}{\alpha^2 \beta} & \frac{v_0 \gamma - u_0 \beta}{\alpha^2 \beta} \\ -\frac{\gamma}{\alpha^2 \beta} & \frac{\gamma^2}{\alpha^2 \beta^2} + \frac{1}{\beta^2} & -\frac{\gamma(v_0 \gamma - u_0 \beta)}{\alpha^2 \beta^2} - \frac{v_0}{\beta^2} \\ \frac{v_0 \gamma - u_0 \beta}{\alpha^2 \beta} & -\frac{\gamma(v_0 \gamma - u_0 \beta)}{\alpha^2 \beta^2} - \frac{v_0}{\beta^2} & \frac{(v_0 \gamma - u_0 \beta)^2}{\alpha^2 \beta^2} + \frac{v_0^2}{\beta^2} + 1 \end{bmatrix}. \quad (7)$$

$$\mathbf{v}_{ij} = [h_{i1}h_{j1}, h_{i1}h_{j2} + h_{i2}h_{j1}, h_{i2}h_{j2}, h_{i3}h_{j1} + h_{i1}h_{j3}, h_{i3}h_{j2} + h_{i2}h_{j3}, h_{i3}h_{j3}]^T. \quad (8)$$

4) Therefore, using the elements of each homography H , we assemble V . Here, V is a known matrix and b (or B) is unknown, so we can again apply SVD to solve for B .

5) Once B has been obtained, compute the intrinsic matrix A from B . According to the paper, A can be recovered using the formula in Eq. (9), and therefore each element of A is given as in Eq. (10) (see Appendix B of the paper).

$$\begin{aligned} \mathbf{r}_1 &= \lambda \mathbf{A}^{-1} \mathbf{h}_1, \\ \mathbf{r}_2 &= \lambda \mathbf{A}^{-1} \mathbf{h}_2, \\ \mathbf{r}_3 &= \mathbf{r}_1 \times \mathbf{r}_2, \\ \mathbf{t} &= \lambda \mathbf{A}^{-1} \mathbf{h}_3. \end{aligned} \quad (9)$$

$$\begin{aligned} v_0 &= \frac{B_{12}B_{13} - B_{11}B_{23}}{B_{11}B_{22} - B_{12}^2}, \\ \lambda &= B_{33} - \frac{B_{13}^2 + v_0(B_{12}B_{13} - B_{11}B_{23})}{B_{11}}, \\ \alpha &= \sqrt{\frac{\lambda}{B_{11}}}, \\ \beta &= \sqrt{\frac{\lambda B_{11}}{B_{11}B_{22} - B_{12}^2}}, \\ \gamma &= -\frac{B_{12}\alpha^2\beta}{\lambda}, \\ u_0 &= \frac{\gamma v_0}{\beta} - \frac{B_{13}\alpha^2}{\lambda}. \end{aligned} \quad (10)$$

1.2.3 Extrinsic Parameters

- **Implementation** : `def _extrinsic_from_H()`

- **Method** : Compute $[R \ t]$ from the estimated H and A

- **Key Features**

- **Zhang's formula**: Implement the equation on page 7 of the paper
- **Scale-factor normalization**
- **Rotation refinement**: Project to the nearest rotation matrix via SVD
- **Z-direction correction**: If $t[2] < 0$, flip the sign

- **Detailed Procedure**

1) Since $H = A[r_1 \ r_2 \ t]$ and we know both H and A , we can write it as in Eq. (11) (see Section

3.1 of the paper).

$$\begin{aligned} \mathbf{r}_1 &= \lambda \mathbf{A}^{-1} \mathbf{h}_1, \\ \mathbf{r}_2 &= \lambda \mathbf{A}^{-1} \mathbf{h}_2, \\ \mathbf{r}_3 &= \mathbf{r}_1 \times \mathbf{r}_2, \\ \mathbf{t} &= \lambda \mathbf{A}^{-1} \mathbf{h}_3, \end{aligned} \quad \text{with} \quad \lambda = \frac{1}{\|\mathbf{A}^{-1} \mathbf{h}_1\|} = \frac{1}{\|\mathbf{A}^{-1} \mathbf{h}_2\|}. \quad (11)$$

2) Due to noise, the resulting rotation matrix may not be perfectly orthonormal; therefore, normalize it to the closest valid rotation via SVD and enforce $\det(R) > 0$. (See Appendix C of the paper)

1.2.4 Non-linear Refinement

- **Implementation** : `def refine_all_parameters()`, `class ZhangRefinement`

- **Method** : Perform non-linear optimization using the Maximum Likelihood Estimation in Section 3.2 of the paper. And Estimate radial distortion only (k_1, k_2) ; tangential (p_1, p_2) are fixed to 0. Skew γ is fixed to 0. Because paper mentions only radial distortion.

- **Key Features**

- Rodrigues formula: convert between rotation vectors and rotation matrices.
- Joint refinement of intrinsics + radial distortion + extrinsics.
- Tangential (p_1, p_2) are not optimized (set to 0)

- **Detailed Procedure**

1) **Build the parameter vector**: concatenate 4 intrinsics + 2 distortion terms + 6 extrinsic parameters into a 1D vector. (Why?? `least_squares` accepts a single 1D parameter vector, so we flatten here and later expand inside `def _unpack_params()`.)

2) **Compute residuals**(`def _residual_function()`) : since the objective matches Eq. (13) in the paper (Section 3.2), compute residuals accordingly:

- m_{ij} : observed 2D corner
- $\hat{m}$: reprojected point using the current parameters

$$\sum_{i=1}^n \sum_{j=1}^m \|\mathbf{m}_{ij} - \hat{\mathbf{m}}(\mathbf{A}, \mathbf{R}_i, \mathbf{t}_i, \mathbf{M}_j)\|^2. \quad (13)$$

3) **Distortion modeling**: apply radial distortion as in Eq. (14), Tangential terms (Eq. (15)) are set to zero: $p_1 = p_2 = 0$ (Section 3.3).

$$\tilde{x} = x + x[k_1(x^2 + y^2) + k_2(x^2 + y^2)^2], \quad \tilde{y} = y + y[k_1(x^2 + y^2) + k_2(x^2 + y^2)^2]. \quad (14)$$

$$x_t = 2p_1xy + p_2(r^2 + 2x^2), \quad y_t = p_1(r^2 + 2y^2) + 2p_2xy. \quad (15)$$

4) With these distortions applied, run Levenberg–Marquardt optimization.

⇒ Return value of `def refine_all_parameters()` : intrinsics, extrinsics, and radial parameters (k_1, k_2) are refined: (p_1, p_2) remain 0

1.3 OpenCV

- **Implementation and Code structure**

`class OpenCVCalibrator` → create object : `ZhangCalibrator.calibrate_camera()`

- **Detailed Procedure**

1) `cv2.calibrateCamera()` : Provide, for each image view, the 3D board coordinates on the plane

(Z=0), the detected 2D corner coordinates, and the image size (taken from the first image). The function then optimizes the parameters assuming the standard five distortion coefficients (k1, k2, p1, p2, k3) along with intrinsics and extrinsics. It returns the camera parameters and the overall RMS reprojection error.

2) **Returned Results** : `cv2.calibrateCamera()` returns extrinsics as vectors (rvecs, tvecs) rather than matrices. Therefore, for each view, convert the rotation vector to a rotation matrix using `cv2.Rodrigues()`, keep the translation as a (3×1) column vector, and then assemble the extrinsic parameters from these.

2. Intrinsic & Extrinsic parameter

2.1 Experiment Setup

- **Number of images** : 25 (1080 × 1440)
- **Output**: For each of the 25 images, output the computed extrinsic matrix $[R \ t]$

2.2 Zhang's method

(1) Intrinsic (K)

```
INTRINSICS (K)
-----
fx = 1074.329576   fy = 1071.452105   cx = 540.072327   cy = 723.702517   skew = 0.000000
K =
1074.329576   0.000000   540.072327
  0.000000  1071.452105   723.702517
  0.000000   0.000000   1.000000
```

(2) Extrinsic (per view, $[R \ t]$)

<pre>[View 00] X_c = R_i * X_w + t_i 0.858263 -0.421931 0.292163 -55.702391 0.504139 0.799702 -0.326067 -88.644404 -0.096066 0.427143 0.899066 270.577499</pre>	<pre>[View 06] X_c = R_i * X_w + t_i 0.970661 0.198992 0.134979 -102.463114 -0.156143 0.948532 -0.275510 -28.681543 -0.182856 0.246351 0.951775 424.159817</pre>
<pre>[View 01] X_c = R_i * X_w + t_i -0.945329 0.125063 0.301186 38.747121 -0.217525 -0.929893 -0.296618 165.721001 0.242975 -0.345917 0.906259 298.936756</pre>	<pre>[View 07] X_c = R_i * X_w + t_i 0.833634 0.463100 0.300986 -148.978728 -0.545887 0.773750 0.321431 -126.374969 -0.084034 -0.432260 0.897825 412.423061</pre>
<pre>[View 02] X_c = R_i * X_w + t_i -0.786032 -0.025324 0.617667 35.410148 0.057810 -0.997793 0.032659 121.793843 0.615477 0.061378 0.785761 243.651077</pre>	<pre>[View 08] X_c = R_i * X_w + t_i 0.994466 0.001263 0.105051 -84.225304 -0.043170 0.916519 0.397654 -112.166880 -0.095779 -0.399989 0.911502 349.478144</pre>
<pre>[View 03] X_c = R_i * X_w + t_i 0.915226 0.340447 0.215537 -125.257488 -0.399357 0.837582 0.372787 -94.830244 -0.053616 -0.427261 0.902537 353.660464</pre>	<pre>[View 09] X_c = R_i * X_w + t_i 0.932753 -0.356581 -0.053114 3.430554 0.343829 0.835571 0.428489 -161.171756 -0.108411 -0.417937 0.901984 355.791424</pre>
<pre>[View 04] X_c = R_i * X_w + t_i 0.994523 -0.084887 0.060975 -66.724240 0.049490 0.896325 0.440626 -116.455766 -0.092057 -0.435195 0.895618 351.577205</pre>	<pre>[View 10] X_c = R_i * X_w + t_i 0.865467 0.437826 0.243464 -159.103895 -0.499051 0.711055 0.495327 -96.503436 0.043751 -0.550190 0.833892 385.242472</pre>
<pre>[View 05] X_c = R_i * X_w + t_i 0.991358 0.120857 0.051013 -109.182192 -0.107331 0.970858 -0.214278 -63.926214 -0.075424 0.206951 0.975440 295.168426</pre>	<pre>[View 11] X_c = R_i * X_w + t_i 0.998931 0.046176 0.001996 -91.925508 -0.037662 0.788197 0.614270 -151.086629 0.026791 -0.613689 0.789093 358.970470</pre>

[View 12]	$X_c = R_i * X_w + t_i$			
	-0.760383	0.060888	-0.646614	62.248991
	-0.016139	-0.997060	-0.074910	117.131559
	-0.649274	-0.046525	0.759130	349.721521
[View 13]	$X_c = R_i * X_w + t_i$			
	-0.874537	0.027473	-0.484181	112.030081
	-0.008048	-0.999079	-0.042152	139.751959
	-0.484893	-0.032967	0.873952	340.177012
[View 14]	$X_c = R_i * X_w + t_i$			
	-0.936447	0.015308	-0.350475	101.906858
	0.005623	-0.998264	-0.058626	129.970326
	-0.350764	-0.056870	0.934735	318.706638
[View 15]	$X_c = R_i * X_w + t_i$			
	-0.966110	-0.014912	-0.257699	106.008758
	-0.001236	-0.998051	0.062389	115.514641
	-0.258127	0.060593	0.964209	291.877648
[View 16]	$X_c = R_i * X_w + t_i$			
	0.997287	-0.012349	-0.072568	-57.809227
	0.018605	0.996106	0.086178	-120.172276
	0.071221	-0.087295	0.993633	289.576745
[View 17]	$X_c = R_i * X_w + t_i$			
	0.988274	0.000953	0.152369	-73.913151
	-0.017970	0.998520	0.051328	-123.519666
	-0.151633	-0.053464	0.986990	290.187139

[View 18]	$X_c = R_i * X_w + t_i$			
	-0.949707	-0.013668	0.312841	66.347375
	0.026207	-0.999011	0.035911	120.556243
	0.312041	0.042304	0.949126	235.818555
[View 19]	$X_c = R_i * X_w + t_i$			
	-0.913256	-0.015830	0.407079	41.896637
	0.027871	-0.999331	0.023666	121.318214
	0.406432	0.032958	0.913086	224.856273
[View 20]	$X_c = R_i * X_w + t_i$			
	-0.876551	-0.007670	0.481248	42.007537
	0.005117	-0.999965	-0.006618	123.373911
	0.481282	-0.003338	0.876559	255.548958
[View 21]	$X_c = R_i * X_w + t_i$			
	0.963919	-0.250424	0.090269	-59.931779
	0.265843	0.923085	-0.277924	-92.913493
	-0.013727	0.291894	0.956352	254.349936
[View 22]	$X_c = R_i * X_w + t_i$			
	0.996112	0.086179	0.018279	-77.788219
	-0.072566	0.920304	-0.384414	-40.636033
	-0.049951	0.381593	0.922980	265.853857
[View 23]	$X_c = R_i * X_w + t_i$			
	0.941475	0.336839	0.012817	-100.816092
	-0.334829	0.938893	-0.079805	-77.450935
	-0.038915	0.070843	0.996728	333.038364
[View 24]	$X_c = R_i * X_w + t_i$			
	0.997792	0.012265	0.065274	-70.849474
	-0.013645	0.999692	0.020735	-102.342230
	-0.064999	-0.021579	0.997652	291.459968

(3) Distortion coefficient from Zhang's method (via Non-linear refinement)

Zhang's distortion : [0.15344215429736724 -0.3816973194385432]

k1=0.153442 k2= -0.381697

2.3 OpenCV

(1) Intrinsic (K)

fx = 1074.221800 fy = 1071.612660 cx = 541.892717 cy = 715.572287 skew = 0.000000

K =

1074.221800	0.000000	541.892717
0.000000	1071.612660	715.572287
0.000000	0.000000	1.000000

(2) Extrinsic (Per view, $[R \ t]$)

[View 00]	$X_c = R_i * X_w + t_i$	
0.858580	-0.422666	0.290162 -56.185174
0.503143	0.803326	-0.318615 -86.571986
-0.098427	0.419549	0.902381 271.202334
[View 01]	$X_c = R_i * X_w + t_i$	
-0.945791	0.125696	0.299465 38.219946
-0.215466	-0.932772	-0.288982 168.043199
0.243009	-0.337841	0.909291 297.597004
[View 02]	$X_c = R_i * X_w + t_i$	
-0.787130	-0.025477	0.616260 34.987645
0.062842	-0.997260	0.039038 123.635459
0.613577	0.069456	0.786574 242.668282
[View 03]	$X_c = R_i * X_w + t_i$	
0.915328	0.341301	0.213746 -125.892836
-0.399764	0.834150	0.379977 -92.086838
-0.048610	-0.433251	0.899961 354.108757
[View 04]	$X_c = R_i * X_w + t_i$	
0.994677	-0.084211	0.059380 -67.327717
0.048895	0.893001	0.447391 -113.756157
-0.090702	-0.442107	0.892365 352.255061
[View 05]	$X_c = R_i * X_w + t_i$	
0.991523	0.120458	0.048701 -109.697493
-0.108044	0.972587	-0.205917 -61.663168
-0.072170	0.198910	0.977357 295.444309

[View 12]	$X_c = R_i * X_w + t_i$	
-0.759565	0.060869	-0.647577 61.676764
-0.021177	-0.997398	-0.068911 119.829905
-0.650087	-0.038628	0.758877 348.908083
[View 13]	$X_c = R_i * X_w + t_i$	
-0.873527	0.027872	-0.485977 111.374053
-0.012444	-0.999312	-0.034946 142.427333
-0.486616	-0.024479	0.873273 339.141397
[View 14]	$X_c = R_i * X_w + t_i$	
-0.935787	0.015589	-0.352220 101.321598
0.002461	-0.998709	-0.050742 132.466434
-0.352557	-0.048351	0.934541 317.756750
[View 15]	$X_c = R_i * X_w + t_i$	
-0.965643	-0.014859	-0.259446 105.474821
-0.003680	-0.997482	0.070824 117.790487
-0.259846	0.069346	0.963157 291.024429
[View 16]	$X_c = R_i * X_w + t_i$	
0.997124	-0.012417	-0.074763 -58.286568
0.019491	0.995319	0.094655 -117.953232
0.073237	-0.095840	0.992699 290.418524
[View 17]	$X_c = R_i * X_w + t_i$	
0.988657	0.009879	0.149866 -74.409097
-0.019081	0.998011	0.060085 -121.273326
-0.148975	-0.062263	0.986879 291.082249

[View 06]	$X_c = R_i * X_w + t_i$	
0.971019	0.198363	0.133316 -103.183169
-0.157470	0.950609	-0.267480 -25.466760
-0.179790	0.238735	0.954296 424.259886
[View 07]	$X_c = R_i * X_w + t_i$	
0.833856	0.464275	0.298551 -149.764000
-0.546635	0.769669	0.329849 -123.076899
-0.076645	-0.438245	0.895582 413.026588
[View 08]	$X_c = R_i * X_w + t_i$	
0.994620	0.001924	0.103568 -84.832793
-0.043904	0.913410	0.404666 -109.470039
-0.093822	-0.407036	0.908581 350.110346
[View 09]	$X_c = R_i * X_w + t_i$	
0.932842	-0.355902	-0.056036 2.837516
0.343569	0.831906	0.435767 -158.406141
-0.108474	-0.425754	0.898313 357.011041
[View 10]	$X_c = R_i * X_w + t_i$	
0.865211	0.439096	0.242084 -159.806134
-0.498956	0.706290	0.502193 -93.433783
0.049529	-0.555292	0.830179 385.607786
[View 11]	$X_c = R_i * X_w + t_i$	
0.998875	0.047408	0.000734 -92.576180
-0.037622	0.783090	0.620769 -148.238450
0.028855	-0.620099	0.783993 359.930851
[View 18]	$X_c = R_i * X_w + t_i$	
-0.950333	-0.013657	0.310934 65.924373
0.028810	-0.998607	0.044194 122.353692
0.309898	0.050957	0.949403 234.878224
[View 19]	$X_c = R_i * X_w + t_i$	
-0.914057	-0.015983	0.405271 41.511563
0.031634	-0.998989	0.031950 122.999286
0.404351	0.042024	0.913638 223.814381
[View 20]	$X_c = R_i * X_w + t_i$	
-0.877498	-0.007741	0.479517 41.572349
0.009185	-0.999958	0.000667 125.308697
0.479492	0.004990	0.877532 254.581700
[View 21]	$X_c = R_i * X_w + t_i$	
0.963968	-0.251013	0.088081 -60.365777
0.265662	0.925513	-0.269906 -90.966467
-0.013771	0.283580	0.958850 254.965206
[View 22]	$X_c = R_i * X_w + t_i$	
0.996226	0.085188	0.016648 -78.226695
-0.072685	0.923583	-0.376445 -38.641106
-0.047445	0.373814	0.926290 266.076724
[View 23]	$X_c = R_i * X_w + t_i$	
0.941585	0.336582	0.011420 -101.389846
-0.334942	0.939455	-0.072376 -74.906012
-0.035088	0.064323	0.997312 333.412476
[View 24]	$X_c = R_i * X_w + t_i$	
0.997914	0.012058	0.063422 -71.337168
-0.013936	0.999475	0.029260 -100.108544
-0.063036	-0.030082	0.997558 292.132699

(3) Distortion coefficient from OpenCV

```
OpenCV dist=
[ 0.14950595 -0.35476826 -0.00345644  0.00090287 -0.02840747]
```

k1=0.14950595 k2= -0.35476826 p1=-0.00345644 p2=0.00090287 k3=-0.02840747

3. Reprojection Error

3.1 Metric

I evaluated with two metrics. First, I computed the reprojection error (Euclidean distance) per image using the formula from the lecture notes, then then aggregated it either by a simple average (MeanL2) or by RMS. Initially I implemented only MeanL2, but since Zhang's paper reports RMS and OpenCV also returns RMS by default, I measured both. OpenCV does not provide MeanL2 automatically, so I implemented it manually; for RMS, I used both OpenCV's returned value and my own implementation following Zhang's RMS formula.

(1) MeanL2 – total_mean_l2

- **Meaning:** Simple average of per-corner pixel errors (Euclidean distances).
- **Advantage:** Intuitive ("on average, how many pixels off?").
- **Caution:** Less sensitive to large errors (outliers).

$$\text{MeanL2} = \frac{1}{M} \sum_{i=1}^V \sum_{j=1}^{N_i} d_{i,j}, \quad d_{i,j} = \|\mathbf{r}_{i,j}\|_2 = \sqrt{(\hat{u}_{i,j} - u_{i,j})^2 + (\hat{v}_{i,j} - v_{i,j})^2}.$$

(2) RMS(Root Mean Square) - total_rms_error

- **Meaning:** Square root of the mean of squared errors.
- **Property:** More sensitive to large errors (outliers increase RMS more).
- **Notes:** Corresponds to OpenCV's default "RMS reprojection error."

$$\text{RMS} = \sqrt{\frac{1}{M} \sum_{i=1}^V \sum_{j=1}^{N_i} \|\mathbf{r}_{i,j}\|_2^2} = \sqrt{\frac{1}{M} \sum_{i,j} ((\hat{u}_{i,j} - u_{i,j})^2 + (\hat{v}_{i,j} - v_{i,j})^2)}.$$

3.2 Result

3.2.1 Reprojection Error

(1) Zhang's method (without distortion)

```
Total RMS error = 2.609168299685612
Total Mean L2 error = 2.0667640172359194
```

- **RMS :** 2.609 px
- **MeanL2 :** 2.067 px

(2) Zhang's method (with distortion)

```
Total RMS error = 0.9433746609724885
Total Mean L2 error = 0.8082173789967052
```

- **RMS :** 0.943 px
- **MeanL2 :** 0.808 px

(3) OpenCV

```
OpenCV RMS (cv2.calibrateCamera): 0.925565 px
OpenCV MeanL2 (cv2.calibrateCamera): 0.791863 px
```

- **RMS :** 0.925 px
- **MeanL2 :** 0.792 px

3.2.2 Intrinsic matrix

Zhang's method (without distortion)	Zhang's method (with distortion)	OpenCV
$K =$ 1102.369624 0.000000 543.048354 0.000000 1099.411263 721.776608 0.000000 0.000000 1.000000	$K =$ 1074.329576 0.000000 540.072327 0.000000 1071.452105 723.702517 0.000000 0.000000 1.000000	$K =$ 1074.221800 0.000000 541.892717 0.000000 1071.612660 715.572287 0.000000 0.000000 1.000000
fx = 1102.369624 fy = 1099.411263 cx = 543.048354 cy = 721.776608 skew = 0.000000	fx = 1074.329576 fy = 1071.452105 cx = 540.072327 cy = 723.702517 skew = 0.000000	fx = 1074.221800 fy = 1071.612660 cx = 541.892717 cy = 715.572287 skew = 0.000000

3.2.3 Extrinsic matrix

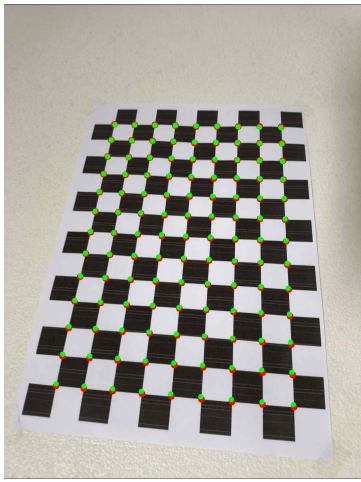
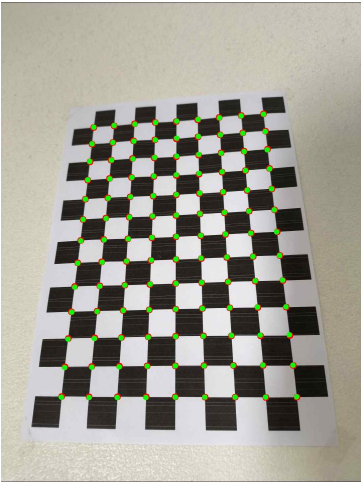
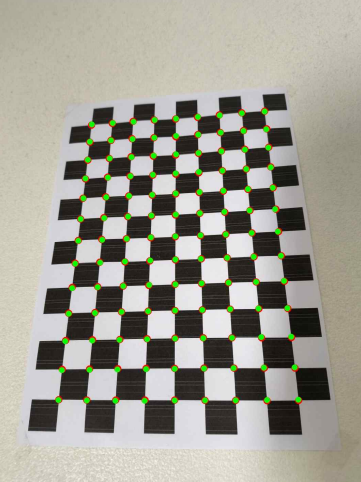
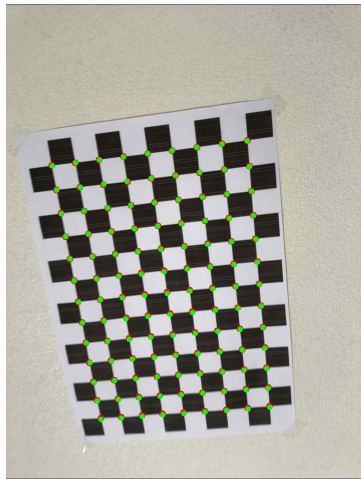
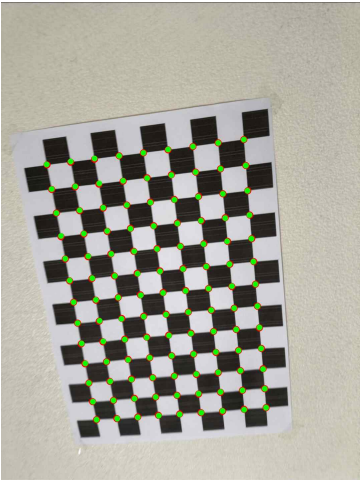
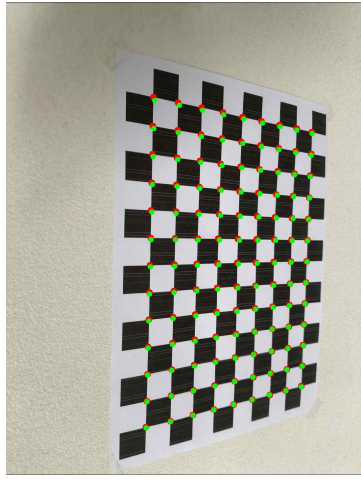
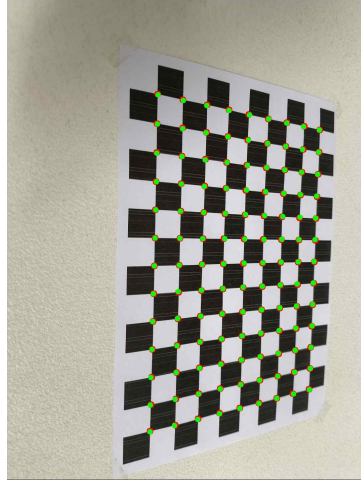
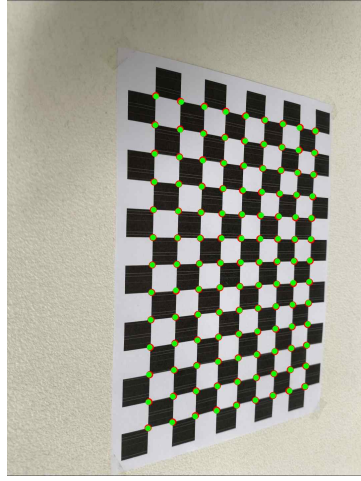
Zhang's method (without distortion)	Zhang's method (with distortion)	OpenCV
[View 00] $X_c = R_i * X_w + t_i$ 0.859879 -0.428074 0.278139 -56.388453 0.503370 0.801704 -0.322318 -88.423883 -0.085009 0.417161 0.904848 275.391300	[View 00] $X_c = R_i * X_w + t_i$ 0.858263 -0.421931 0.292163 -55.702391 0.504139 0.799702 -0.326067 -88.644404 -0.096066 0.427143 0.899066 270.577499	[View 00] $X_c = R_i * X_w + t_i$ 0.858580 -0.422666 0.290162 -56.185174 0.503143 0.803326 -0.318615 -86.571986 -0.098427 0.419549 0.902381 271.202334
[View 01] $X_c = R_i * X_w + t_i$ -0.946604 0.127007 0.296327 38.091923 -0.217295 -0.930336 -0.295394 166.448910 0.238167 -0.344011 0.908258 303.019175	[View 01] $X_c = R_i * X_w + t_i$ -0.945329 0.125063 0.301186 38.747121 -0.217525 -0.929893 -0.296618 165.721001 0.242975 -0.345917 0.906259 298.936756	[View 01] $X_c = R_i * X_w + t_i$ -0.945791 0.125696 0.299465 38.219946 -0.215466 -0.932772 -0.288982 168.043199 0.243009 -0.337841 0.909291 297.597004

All solutions follow $X_c = R X_w + t$ with $R^T R \approx I$ and $\det(R) \approx +1$ across views. OpenCV and Zhang(+dist) extrinsics are closely aligned, while the distortion model reduces edge residuals (see Sec. 3.2.1). Full 25-view matrices are provided in Sec. 2.2–2.3 for Zhang(+dist) and OpenCV; the no-distortion baseline is shown only for representative views.

4. Visualization

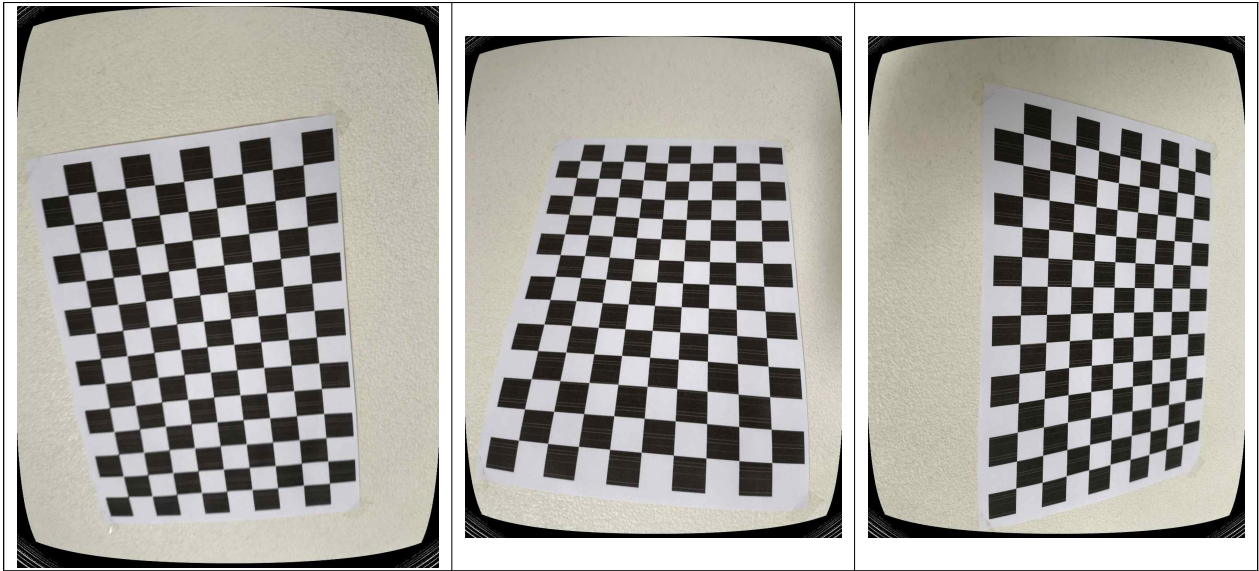
4.1 Reprojection image

- **Red:** actual observed corner locations (**Observed corners**)
- **Green:** predicted locations projected using the estimated camera parameters (**Reprojected corners**)

Zhang's method (without distortion)	Zhang's method (with distortion)	OpenCV
		
		
		

4.2 undistorted image

I generated the undistorted images using the distortion coefficients and camera parameters estimated by OpenCV's calibration. Since `cv2.calibrateCamera()` applies a standard distortion model with non-linear optimization over all images/corners, I expected it to be more accurate than my manual Zhang's-method implementation. (Indeed, both the RMS and MeanL2 results are slightly better with OpenCV.)



5. Compare and Analysis

5.1 Qualitative Compare

See detailed numbers in **3.2 Results**.

5.1.1 Effect of introducing a distortion model (Zhang: without-distortion → with distortion)

- **RMS:** 2.609 → 0.943 (-63.9%)
- **MeanL2:** 2.067 → 0.808 (-60.9%)
- **Interpretation:** Pure homography (projective geometry) cannot account for lens distortion; adding radial/tangential distortion drastically reduces reprojection error.

5.1.2 OpenCV vs. Zhang(+distortion)

- **RMS:** 0.943 → 0.925 (-1.9%)
- **Mean L2:** 0.808 → 0.792 (-2.0%)
- **Interpretation:** The two methods are essentially on par, with OpenCV showing slightly lower errors.

Side note: In all cases, $RMS > MeanL2$, reflecting RMS's greater sensitivity to large errors (outliers).

5.2 Quantitative Compare

- Zhang (without distortion)

- Good near the board center, but residuals systematically increase toward the edges.
- Classic radial distortion pattern (pushing/pulling outward), i.e., consequences of not modeling distortion.

- Zhang (with distortion)

- Edge residuals drop substantially; overall observed–reprojected alignment is denser.
- Minor residuals remain at a few corner points near the image boundary.

- OpenCV

- Similar alignment quality to Zhang (+distortion).
- Residuals at corners/edges appear slightly more uniform, consistent with the marginally better quantitative scores.

5.3 Analysis

(1) Distortion model differences

In my Zhang non-linear refinement, I estimate radial terms only (k_1, k_2) and keep tangential terms fixed ($p_1 = p_2 = 0$). By contrast, OpenCV's default uses a five-coefficient model $[k_1, k_2, p_1, p_2, k_3]$. The absence of p_1, p_2 and k_3 in my model likely explains why OpenCV achieves slightly lower edge residuals and a marginally better overall RMS.

(2) Optimization engine & numerical stability

OpenCV is a widely used, highly tuned library employing mature bundle-adjustment practices (analytic Jacobians, parameter scaling, stopping criteria, max iterations). While my Zhang implementation follows the paper faithfully, we cannot rule out differences in convergence conditions/initialization/normalization versus OpenCV. Such nuances can yield 0.0x-level gaps in the final metrics.